

Title:

Product Performance Analysis in Big Basket Using MySQL Analytical Functions.

Problem Statement:

Big Basket is one of the leading online grocery platforms offering thousands of products across multiple categories. As the product catalogue and customer base grow, the company faces the challenge of making sense of vast amounts of product and pricing data. The business needs to understand how products perform within their categories, how prices compare across similar items, and which categories contribute most to total sales value. Additionally, the operations team requires streamlined query access to product data, the ability to dynamically update pricing, and tools to filter products by category and subcategory for targeted decision-making.

Objective:

To analyze product sales performance and customer preferences using MySQL's advanced SQL functionalities, including ranking, lead and lag analysis, subqueries, views, stored procedures, and common table expressions (CTE).

Dataset Overview

The dataset contains product details for various items sold by Big Basket, stored in the `big_basket` table under the `product_performance_analysis` schema. Each row represents a unique product listing. The key columns include:

Column	Description
id	Unique identifier for each product
product	Name of the product
category	Category of the product (e.g., Baby Care, Beauty & Hygiene, Beverages)
sub_category	Subcategory the product belongs to
brand	Brand of the product
sale_price	Discounted sale price of the product

Column	Description
market_price	Original market price of the product
type	Type or variety of the product
rating	Customer rating for the product (scale of 1 to 5)

Task:

1. Rank products based on their sale price within each category.
2. Find the next product's sale price for comparison within the same category.
3. Create a CTE to get the top 5 highest-rated products for each category.
4. Create a view to simplify querying for product information in the 'Beauty & Hygiene' category.
5. Write a stored procedure to update the sale price of a product by its id.
6. Create a CTE to show the total sales price per category and rank them.
7. Create a view to store the sale price and market price difference for each product.
8. Create a stored procedure to return products from a specific category and sub category.
9. Use CASE to assign discounts based on the market price of the products(>500 as high discount, from 200 to 500 as medium discount or else low discount).